UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍ A EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026

INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	SW-AyLP-APE-04: Clases Objetos y Métodos
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - B
Alumnos participantes:	Ashanga Yumbo Brishy Anahy Calderón Carvajal Fher Dorian Casillas Ochoa Antony Sebastián Paredes Acosta Dereck Shair Sánchez Bastidas Karina Paola ……..
Asignatura:	Algoritmos y lógica de programación
Docente:	Ing. José Rubén Caiza Caizabuano, M.Sc.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:
Desarrollar programas utilizando clases, objetos y métodos, lo cual permite disminuir errores reutilizar el código.

Específicos:

- Definir clases con sus atributos y constructores para modelar entidades del problema, como Autor, Libro, Producto y Cuenta.
- Implementar métodos que realicen operaciones y cálculos sobre los datos de los objetos, como mostrar información, calcular áreas, registrar ventas y manejar saldos.
- Relacionar varias clases entre sí mediante la creación e interacción de objetos en Java, como en el sistema de préstamos de libros.

2.2 Modalidad

Presencial

2.3 Tiempo de duración Presenciales: 6

No presenciales: 0

2.4 Instrucciones

- La tarea se debe realizar de forma individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrolle lo solicitado.
- Codifique en Java cada uno de los ejercicios planteados.
- Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado, adicionalmente realizar capturas de cada una de las clases y métodos de los ejercicios. Las capturas se subirán en un archivo pdf en el formato de Guías APE.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- Computador o laptop.

- Plataforma educativa de la UTA.
- Internet para consultas.
- Papel - Material de escritorio.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas

☐Simuladores y laboratorios virtuales

☐Aplicaciones educativas

☐Recursos audiovisuales

☐Gamificación

☒Inteligencia Artificial

Otros (Especifique): Visual Studio Code, NetBeans, GitHub

2.6 Actividades por desarrollar

A continuación se presentan los cinco ejercicios resueltos.

La APE 4 tiene 18 ejercicios divididos en tres grupos. El Grupo 1 trata casos de la vida real con arreglos de objetos, el Grupo 2 los fundamentos de métodos (tipos de retorno, void, parámetros, paso por valor) y el Grupo 3 objetos que colaboran entre sí. Todo el código está en el repositorio del grupo en GitHub, organizado en carpetas Grupo 1, Grupo 2 y Grupo 3, y cada ejercicio tiene su carpeta con el archivo .java.

En las figuras de esta sección se muestra cada clase por separado, con los números de línea del archivo original, y luego la salida en consola. Los ejercicios se ejecutan uno por uno porque varios repiten nombres de clase (Producto, Libro, Cuenta y Calculadora). Las capturas tomadas por cada integrante al probar su código están en los anexos.

Tabla 1. *Roles y aporte de cada integrante*

Integrante	Rol	Aporte en el repositorio
Paredes Acosta Dereck Shair	Análisis	Grupo 2, ejercicios 3 al 7 (código y PDF de cada uno); README del repositorio
Ashanga Yumbo Brishy Anahy	Diseño	Creación del repositorio; carga del Grupo 1 con sus capturas e infografía
Calderón Carvajal Fher Dorian	Programación	Revisión y ejecución de los 18 ejercicios, capturas del informe e informe final
Casillas Ochoa Antony Sebastián	Pruebas	Grupo 3, ejercicios 1 al 5 con capturas, infografía e informe del grupo
Sánchez Bastidas Karina Paola	Documentación	Código del Grupo 1 en Java y C++; Grupo 2, ejercicios 1, 2 y 8 e infografía del Grupo 2

Grupo 1: Casos de la vida real

Ejercicio 1: Notas del curso

Enunciado: Crear la clase Estudiante con los atributos nombre, nota1, nota2, nota3 y promedio. En main, crear 5 instancias, calcular el promedio de cada una e imprimirlas con el formato nombre : promedio, agregando Aprobado (7 o más), Supletorio (de 5 a 6.99) o Reprobado (menos de 5). Al final, imprimir cuántos estudiantes aprobaron.

Archivo: Grupo 1/Ejercicio 1/NotasCurso.java

NotasCurso.java

```java
/*
 * Guia APE 4 - Ejercicio 1: Notas del curso
 * Estudiante: Paola Sanchez
 */

class Estudiante {
    String nombre;
    double nota1, nota2, nota3;
    double promedio;

    Estudiante(String nombre, double nota1, double nota2, double nota3) {
        this.nombre = nombre;
        this.nota1 = nota1;
        this.nota2 = nota2;
        this.nota3 = nota3;
    }

    void calcularPromedio() {
        promedio = (nota1 + nota2 + nota3) / 3;
        promedio = Math.round(promedio * 100) / 100.0;   // redondeo a 2 decimales
    }

    String estado() {
        if (promedio >= 7) return "Aprobado";
        if (promedio >= 5) return "Supletorio";
        return "Reprobado";
    }
}
```

Figura 1. *Clase Estudiante (NotasCurso.java)*

NotasCurso.java

```java
public class NotasCurso {
    public static void main(String[] args) {
        Estudiante[] lista = {
            new Estudiante("Valeria Castro", 9.5, 8, 9),
            new Estudiante("Javier Yanez", 5, 6, 4.5),
            new Estudiante("Gabriela Nunez", 3, 4, 5.5),
            new Estudiante("Kevin Moreta", 7, 7, 7),
            new Estudiante("Lucia Haro", 8, 6, 7.5)
        };

        int totalAprobados = 0;

        for (int i = 0; i < lista.length; i++) {
            lista[i].calcularPromedio();
            System.out.println(lista[i].nombre + " : " + lista[i].promedio + " " + lista[i].estado());

            if (lista[i].estado().equals("Aprobado")) {
                totalAprobados++;
            }
        }

        System.out.println("============================");
        System.out.println("Total de aprobados: " + totalAprobados);
    }
}
```

Figura 2. *Clase NotasCurso (NotasCurso.java)*

Output - Ejercicios_APE_4 (run)

```
run:
Valeria Castro : 8.83 Aprobado
Javier Yanez : 5.17 Supletorio
Gabriela Nunez : 4.17 Reprobado
Kevin Moreta : 7.0 Aprobado
Lucia Haro : 7.17 Aprobado
============================
Total de aprobados: 3
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 3. *Salida en consola, Grupo 1, ejercicio 1*

El promedio y el estado se calculan con métodos de la clase Estudiante, así el main solo recorre el arreglo. El promedio se redondea a dos decimales con Math.round, si no salían valores como 8.833333. Aprobaron 3 de 5.

Ejercicio 2: Inventario de una tienda

Enunciado: Crear la clase Producto con nombre, precio y stock. Crear 4 productos e imprimirlos con el formato nombre : precio. Simular la venta de 3 unidades de dos de ellos, mostrar "Reponer" junto a cada producto que quede con menos de 5 unidades e imprimir el valor total del inventario (suma de precio por stock).

Archivo: Grupo 1/Ejercicio 2/InventarioTienda.java

InventarioTienda.java

```java
/*
 * Guia APE 4 - Ejercicio 2: Inventario de una tienda
 * Estudiante: Paola Sanchez
 */

class Producto {
    String nombre;
    double precio;
    int stock;

    Producto(String nombre, double precio, int stock) {
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    void vender(int cantidad) {
        if (cantidad <= stock) {
            stock -= cantidad;
            System.out.println("Venta: " + cantidad + " unidades de " + nombre);
        } else {
            System.out.println("No hay suficiente stock de " + nombre);
        }
    }

    double valorEnStock() {
        return precio * stock;
    }
}
```

Figura 4. *Clase Producto (InventarioTienda.java)*

InventarioTienda.java

```java
public class InventarioTienda {
    public static void main(String[] args) {
        Producto[] productos = {
            new Producto("Cuaderno", 2.50, 12),
            new Producto("Esfero", 0.60, 30),
            new Producto("Borrador", 0.35, 6),
            new Producto("Carpeta", 1.75, 4)
        };

        System.out.println(">> Lista de productos");
        for (Producto p : productos) {
            System.out.println(p.nombre + " : " + p.precio);
        }

        System.out.println();
        productos[1].vender(3);
        productos[2].vender(3);

        System.out.println();
        System.out.println(">> Estado del stock");
        double valorTotal = 0;
        for (Producto p : productos) {
            String aviso = (p.stock < 5) ? " -> Reponer" : "";
            System.out.println(p.nombre + " : " + p.precio + " (quedan " + p.stock + ")" + aviso);
            valorTotal += p.valorEnStock();
        }

        valorTotal = Math.round(valorTotal * 100) / 100.0;
        System.out.println();
        System.out.println("Valor total del inventario: $" + valorTotal);
    }
}
```

Figura 5. *Clase InventarioTienda (InventarioTienda.java)*

```
Output - Ejercicios_APE_4 (run)
run:
>> Lista de productos
Cuaderno : 2.5
Esfero : 0.6
Borrador : 0.35
Carpeta : 1.75

Venta: 3 unidades de Esfero
Venta: 3 unidades de Borrador

>> Estado del stock
Cuaderno : 2.5 (quedan 12)
Esfero : 0.6 (quedan 27)
Borrador : 0.35 (quedan 3) -> Reponer
Carpeta : 1.75 (quedan 4) -> Reponer

Valor total del inventario: $54.25
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 6. *Salida en consola, Grupo 1, ejercicio 2*

Se venden 3 esferos y 3 borradores con el método vender(), que primero revisa si alcanza el stock. Al borrador le quedan 3 y la carpeta ya tenía 4, por eso los dos salen con Reponer. El valor total da $54.25.

Ejercicio 3: Veterinaria

Enunciado: Crear la clase Mascota con nombre, especie, edad y peso. Crear 3 mascotas e imprimirlas con el formato nombre : especie : peso, mostrando "Revisar peso" si pesan más de 20 kg. Luego escribir Mascota m4 = m1;, sumar 1 a m4.edad, imprimir m1.edad y explicar en un comentario por qué cambió.

Archivo: Grupo 1/Ejercicio 3/Veterinaria.java

Veterinaria.java

```java
/*
 * Guia APE 4 - Ejercicio 3: Veterinaria
 * Estudiante: Paola Sanchez
 */

class Mascota {
    String nombre;
    String especie;
    int edad;
    double peso;

    Mascota(String nombre, String especie, int edad, double peso) {
        this.nombre = nombre;
        this.especie = especie;
        this.edad = edad;
        this.peso = peso;
    }

    void mostrar() {
        String alerta = (peso > 20) ? "   <-- Revisar peso" : "";
        System.out.println(nombre + " : " + especie + " : " + peso + alerta);
    }
}
```

Figura 7. *Clase Mascota (Veterinaria.java)*

Veterinaria.java

```java
public class Veterinaria {
    public static void main(String[] args) {
        Mascota m1 = new Mascota("Toby", "Perro", 4, 22.3);
        Mascota m2 = new Mascota("Nala", "Gato", 2, 3.8);
        Mascota m3 = new Mascota("Coco", "Conejo", 1, 1.9);

        System.out.println("Registro de mascotas (peso en kg)");
        m1.mostrar();
        m2.mostrar();
        m3.mostrar();

        System.out.println();
        System.out.println("m1.edad al inicio = " + m1.edad);

        Mascota m4 = m1;
        m4.edad = m4.edad + 1;

        System.out.println("m1.edad luego de modificar m4 = " + m1.edad);

        /*
         * Explicacion: m1.edad paso de 4 a 5 aunque nunca se toco m1.
         * Con "Mascota m4 = m1;" no se hace "new", por lo tanto no se crea
         * otro objeto. Lo que se copia es la referencia, es decir m4 y m1
         * quedan apuntando al mismo objeto en memoria (el de Toby).
         * Cualquier cambio hecho a traves de m4 se ve tambien desde m1
         * porque en realidad es el mismo objeto con dos nombres.
         */
    }
}
```

Figura 8. *Clase Veterinaria (Veterinaria.java)*

Output - Ejercicios_APE_4 (run)

```
run:
Registro de mascotas (peso en kg)
Toby : Perro : 22.3   <-- Revisar peso
Nala : Gato : 3.8
Coco : Conejo : 1.9

m1.edad al inicio = 4
m1.edad luego de modificar m4 = 5
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 9. *Salida en consola, Grupo 1, ejercicio 3*

La edad de m1 pasa de 4 a 5 aunque nunca se modificó m1 directamente. Con Mascota m4 = m1 no se crea otra mascota porque no hay new, solo se copia la referencia, entonces m1 y m4 son el mismo objeto.

Ejercicio 4: Cuenta de ahorro

Enunciado: Crear la clase Cuenta con titular y saldo. Crear 2 cuentas, una con 200 y otra con 350. Con un while, depositar 50 por mes en cada cuenta hasta que llegue a 500, imprimiendo cada mes con el formato titular : saldo, y al final cuántos meses necesitó cada una (6 y 3).

Archivo: Grupo 1/Ejercicio 4/CuentaAhorro.java

CuentaAhorro.java

```java
/*
 * Guia APE 4 - Ejercicio 4: Cuenta de ahorro
 * Estudiante: Paola Sanchez
 */

class Cuenta {
    String titular;
    double saldo;

    Cuenta(String titular, double saldo) {
        this.titular = titular;
        this.saldo = saldo;
    }

    void depositar(double monto) {
        saldo += monto;
    }
}
```

Figura 10. *Clase Cuenta (CuentaAhorro.java)*

CuentaAhorro.java

```java
public class CuentaAhorro {
    public static void main(String[] args) {
        Cuenta cuentaA = new Cuenta("Sofia Guaman", 200);
        Cuenta cuentaB = new Cuenta("Mateo Lema", 350);

        final double META = 500;
        final double DEPOSITO = 50;

        int mesesA = 0;
        System.out.println("Ahorro de " + cuentaA.titular + " (inicia con " + cuentaA.saldo + ")");
        while (cuentaA.saldo < META) {
            cuentaA.depositar(DEPOSITO);
            mesesA++;
            System.out.println("  Mes " + mesesA + " -> " + cuentaA.titular + " : " + cuentaA.saldo);
        }

        int mesesB = 0;
        System.out.println("Ahorro de " + cuentaB.titular + " (inicia con " + cuentaB.saldo + ")");
        while (cuentaB.saldo < META) {
            cuentaB.depositar(DEPOSITO);
            mesesB++;
            System.out.println("  Mes " + mesesB + " -> " + cuentaB.titular + " : " + cuentaB.saldo);
        }

        System.out.println();
        System.out.println("Resumen:");
        System.out.println(cuentaA.titular + " llego a la meta en " + mesesA + " meses");
        System.out.println(cuentaB.titular + " llego a la meta en " + mesesB + " meses");
    }
}
```

Figura 11. *Clase CuentaAhorro (CuentaAhorro.java)*

Output - Ejercicios_APE_4 (run)

```
run:
Ahorro de Sofia Guaman (inicia con 200.0)
  Mes 1 -> Sofia Guaman : 250.0
  Mes 2 -> Sofia Guaman : 300.0
  Mes 3 -> Sofia Guaman : 350.0
  Mes 4 -> Sofia Guaman : 400.0
  Mes 5 -> Sofia Guaman : 450.0
  Mes 6 -> Sofia Guaman : 500.0
Ahorro de Mateo Lema (inicia con 350.0)
  Mes 1 -> Mateo Lema : 400.0
  Mes 2 -> Mateo Lema : 450.0
  Mes 3 -> Mateo Lema : 500.0

Resumen:
Sofia Guaman llego a la meta en 6 meses
Mateo Lema llego a la meta en 3 meses
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 12. *Salida en consola, Grupo 1, ejercicio 4*

El while se repite mientras el saldo sea menor a la meta y en cada vuelta llama a depositar(). Sofía necesitó 6 meses y Mateo 3, que coincide con lo que pedía el enunciado.

Ejercicio 5: Biblioteca

Enunciado: Crear las clases Libro (titulo, autor, disponible) y Socio (nombre, librosPrestados). Crear 4 libros y 2 socios, registrar que el primer socio presta 2 libros y el segundo 1, y al final imprimir cada libro como Disponible o Prestado, cada socio con sus libros prestados y cuántos libros quedan disponibles.

Archivo: Grupo 1/Ejercicio 5/Biblioteca.java

Biblioteca.java

```java
/*
 * Guia APE 4 - Ejercicio 5: Biblioteca
 * Estudiante: Paola Sanchez
 */

class Libro {
    String titulo;
    String autor;
    boolean disponible;

    Libro(String titulo, String autor) {
        this.titulo = titulo;
        this.autor = autor;
        this.disponible = true;   // todo libro nuevo empieza disponible
    }
}
```

Figura 13. *Clase Libro (Biblioteca.java)*

Biblioteca.java

```java
class Socio {
    String nombre;
    int librosPrestados;

    Socio(String nombre) {
        this.nombre = nombre;
        this.librosPrestados = 0;
    }

    void prestar(Libro libro) {
        if (libro.disponible) {
            libro.disponible = false;
            librosPrestados++;
        }
    }
}
```

Figura 14. *Clase Socio (Biblioteca.java)*

Biblioteca.java

```java
public class Biblioteca {
    public static void main(String[] args) {
        Libro[] libros = {
            new Libro("El Alquimista", "Paulo Coelho"),
            new Libro("Rayuela", "Julio Cortazar"),
            new Libro("Ficciones", "Jorge Luis Borges"),
            new Libro("Pedro Paramo", "Juan Rulfo")
        };

        Socio socio1 = new Socio("Camila Ortiz");
        Socio socio2 = new Socio("Diego Pilco");

        socio1.prestar(libros[0]);
        socio1.prestar(libros[1]);
        socio2.prestar(libros[2]);

        System.out.println("*** Estado de los libros ***");
        int librosLibres = 0;
        for (Libro l : libros) {
            System.out.println(l.titulo + " : " + (l.disponible ? "Disponible" : "Prestado"));
            if (l.disponible) {
                librosLibres++;
            }
        }

        System.out.println("*** Socios ***");
        System.out.println(socio1.nombre + " : " + socio1.librosPrestados);
        System.out.println(socio2.nombre + " : " + socio2.librosPrestados);

        System.out.println("Quedan " + librosLibres + " libro(s) disponible(s)");
    }
}
```

Figura 15. *Clase Biblioteca (Biblioteca.java)*

Output - Ejercicios_APE_4 (run)

```
run:
*** Estado de los libros ***
El Alquimista : Prestado
Rayuela : Prestado
Ficciones : Prestado
Pedro Paramo : Disponible
*** Socios ***
Camila Ortiz : 2
Diego Pilco : 1
Quedan 1 libro(s) disponible(s)
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 16. *Salida en consola, Grupo 1, ejercicio 5*

El método prestar() de Socio recibe el libro, lo marca como no disponible y suma uno al contador del socio. Como el libro llega por referencia, el cambio se ve en el arreglo del main. Solo queda disponible Pedro Páramo.

Grupo 2: Fundamentos de métodos

Ejercicio 1: Identificación de tipos de retorno

Enunciado: Dado el método public int calcularArea(double base, double altura), que calcula el área y la imprime, explicar por qué genera un error de compilación en Java y cómo se corregiría.

Archivo: Grupo 2/Ejercicio 1/Area.java

Area.java

```java
public class Area {
    // El original declara int, pero no devuelve un valor.
    // Se usa double para conservar los decimales
    // y return para devolver el resultado.
    public double calcularArea(double base, double altura) {
        double area = base * altura;
        return area;
    }

    public static void main(String[] args) {
        Area figura = new Area();
        double resultado = figura.calcularArea(4.5, 3.0);
        System.out.println("El área es: " + resultado);
    }
}
```

Figura 17. *Clase Area (Area.java)*

Output - Ejercicios_APE_4 (run)

```
run:
El área es: 13.5
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 18. *Salida en consola, Grupo 2, ejercicio 1*

El error es que la firma dice int pero el método no tiene return, y además el área es double. Se cambió el tipo de retorno a double y se agregó return area, así el resultado se puede usar en el main. Con 4.5 y 3.0 da 13.5.

Ejercicio 2: Métodos void

Enunciado: Crear un método dentro de la clase Bienvenida que reciba un nombre (String) y una edad (int) e imprima "Hola [nombre], tienes [edad] años. ¡Bienvenido al sistema!". ¿Qué tipo de retorno debe llevar y por qué?

Archivo: Grupo 2/Ejercicio 2/Bienvenida.java

Bienvenida.java

```java
public class Bienvenida {
    // Usa void porque imprime y no devuelve un valor.
    public void saludar(String nombre, int edad) {
        System.out.println("Hola " + nombre + ", tienes "
                + edad + " años. ¡Bienvenido al sistema!");
    }

    public static void main(String[] args) {
        Bienvenida bienvenida = new Bienvenida();
        bienvenida.saludar("Antony", 19);
    }
}
```

Figura 19. *Clase Bienvenida (Bienvenida.java)*

Output - Ejercicios_APE_4 (run)

```
run:
Hola Antony, tienes 19 años. ¡Bienvenido al sistema!
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 20. *Salida en consola, Grupo 2, ejercicio 2*

Lleva void porque el método solo imprime el saludo y no tiene que devolver ningún valor a quien lo llama.

Ejercicio 3: Parámetros y argumentos

Enunciado: En el método multiplicar(double x, double y) y la llamada miObjeto.multiplicar(4.0, 5.5), identificar cuáles son los parámetros y cuáles los argumentos, explicando la diferencia.

Archivo: Grupo 2/Ejercicio 3/Calculadora.java

Calculadora.java

```java
public class Calculadora {
    // x e y son parámetros: reciben los datos del método.
    public double multiplicar(double x, double y) {
        return x * y;
    }

    public static void main(String[] args) {
        Calculadora miObjeto = new Calculadora();
        // 4.0 y 5.5 son argumentos: valores de la llamada.
        double res = miObjeto.multiplicar(4.0, 5.5);
        System.out.println("Resultado: " + res);
    }
}
```

Figura 21. *Clase Calculadora (Calculadora.java)*

Output - Ejercicios_APE_4 (run)

```
run:
Resultado: 22.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 22. *Salida en consola, Grupo 2, ejercicio 3*

x e y son los parámetros: las variables que se declaran en la firma para recibir datos. 4.0 y 5.5 son los argumentos: los valores que se mandan al momento de llamar al método. El resultado es 22.0.

Ejercicio 4: Paso por valor

Enunciado: Si la variable numero vale 10 en el main y se llama a incrementar(numero), donde el método hace n = n + 5, ¿cuál es el valor impreso después? Explicar por qué.

Archivo: Grupo 2/Ejercicio 4/PasoPorValor.java

PasoPorValor.java

```java
public class PasoPorValor {
    public void incrementar(int n) {
        n = n + 5;
    }

    public static void main(String[] args) {
        PasoPorValor ejemplo = new PasoPorValor();
        int numero = 10;
        ejemplo.incrementar(numero);
        // Imprime 10: n recibe una copia del valor.
        // Cambiar n no modifica numero en el main.
        System.out.println(numero);
    }
}
```

Figura 23. *Clase PasoPorValor (PasoPorValor.java)*

Output - Ejercicios_APE_4 (run)

```
run:
10
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 24. *Salida en consola, Grupo 2, ejercicio 4*

Se imprime 10. Java pasa los int por valor, entonces n recibe una copia y lo que se suma dentro del método no cambia la variable del main. Es diferente a la veterinaria del Grupo 1, donde m1 y m4 compartían el mismo objeto.

Ejercicio 5: Método con múltiples parámetros

Enunciado: Agregar a la clase Calculadora un método calcularPromedio que reciba tres notas double (n1, n2, n3), calcule el promedio sumándolas y dividiéndolas entre 3, y retorne el resultado.

Archivo: Grupo 2/Ejercicio 5/Calculadora (1).java

Calculadora (1).java

```java
public class Calculadora {
    public double calcularPromedio(double n1,
                                   double n2,
                                   double n3) {
        double promedio = (n1 + n2 + n3) / 3;
        return promedio;
    }

    public static void main(String[] args) {
        Calculadora calculadora = new Calculadora();
        double resultado =
                calculadora.calcularPromedio(8, 9, 10);
        System.out.println("Promedio: " + resultado);
    }
}
```

Figura 25. *Clase Calculadora (Calculadora (1).java)*

Output - Ejercicios_APE_4 (run)

```
run:
Promedio: 9.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 26. *Salida en consola, Grupo 2, ejercicio 5*

El método recibe las tres notas y devuelve el promedio con return, por eso en el main se guarda en una variable antes de imprimirlo. Con 8, 9 y 10 da 9.0. En el repositorio el archivo quedó como "Calculadora (1).java"; para compilarlo hay que renombrarlo a Calculadora.java.

Ejercicio 6: Validación de datos
Enunciado: Crear un método verificarEdad(int edad). Si la edad es menor a 0 o mayor a 120 debe imprimir "Edad inválida" y retornar false. Si es correcta, debe retornar true.
Archivo: Grupo 2/Ejercicio 6/ValidacionEdad.java

ValidacionEdad.java

```java
public class ValidacionEdad {
    public boolean verificarEdad(int edad) {
        if (edad < 0 || edad > 120) {
            System.out.println("Edad inválida");
            return false;
        }
        return true;
    }

    public static void main(String[] args) {
        ValidacionEdad validacion = new ValidacionEdad();
        int edad = 19;
        boolean esValida = validacion.verificarEdad(edad);
        System.out.println("Edad válida: " + esValida);
    }
}
```

Figura 27. *Clase ValidacionEdad (ValidacionEdad.java)*

Output - Ejercicios_APE_4 (run)

```
run:
Edad válida: true
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 28. *Salida en consola, Grupo 2, ejercicio 6*

Con edad 19 el método devuelve true. Cambiando el valor en el main también probamos -1 y 121, que imprimen Edad inválida y devuelven false, y los límites 0 y 120, que son válidos.

Ejercicio 7: Completar la clase Producto
Enunciado: Rellenar los espacios vacíos de la clase Producto para que calcule el valor total del inventario de un producto (stock * precioUnitario).
Archivo: Grupo 2/Ejercicio 7/Producto.java

Producto.java

```java
public class Producto {
    String nombre;
    int stock;
    double precioUnitario;

    public double calcularValorTotal() {
        return stock * precioUnitario;
    }

    public static void main(String[] args) {
        Producto producto = new Producto();
        producto.stock = 10;
        producto.precioUnitario = 25.5;
        double total = producto.calcularValorTotal();
        System.out.println("Valor del inventario: " + total);
    }
}
```

Figura 29. *Clase Producto (Producto.java)*

Output - Ejercicios_APE_4 (run)

```
run:
Valor del inventario: 255.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 30. *Salida en consola, Grupo 2, ejercicio 7*

Los espacios eran double, como tipo de retorno, y return. Con 10 unidades a 25.5 el valor es 255.0.

Ejercicio 8: Instanciación e invocación en el main
Enunciado: Escribir el código del main de una clase principal para: a) instanciar un Producto llamado p, b) asignarle stock = 10 y precioUnitario = 25.5, c) llamar a calcularValorTotal(), guardar el resultado en totalInventario e imprimirlo.
Archivo: Grupo 2/Ejercicio 8/Principal.java

Principal.java

```java
class Producto {
    String nombre;
    int stock;
    double precioUnitario;

    public double calcularValorTotal() {
        return stock * precioUnitario;
    }
}
```

Figura 31. *Clase Producto (Principal.java)*

Principal.java

```java
public class Principal {
    public static void main(String[] args) {
        Producto p = new Producto();
        p.stock = 10;
        p.precioUnitario = 25.5;
        double totalInventario = p.calcularValorTotal();
        System.out.println("Total del inventario: "
                + totalInventario);
    }
}
```

Figura 32. *Clase Principal (Principal.java)*

Output - Ejercicios_APE_4 (run)

```
run:
Total del inventario: 255.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 33. *Salida en consola, Grupo 2, ejercicio 8*

En el mismo archivo están la clase Producto y la clase Principal con el main, que sigue los tres pasos del enunciado. El total es 255.0, igual que en el ejercicio 7.

Grupo 3: Objetos que colaboran

Ejercicio 1: Libro y Autor (objeto como atributo)

Enunciado: Crear la clase Autor (nombre, nacionalidad) y la clase Libro (titulo, anio, autor). Libro debe tener un atributo de tipo Autor y un método mostrarInformacion() que imprima el título, el año y el nombre del autor. En main, crear 2 libros.
Archivo: Grupo 3/Ejercicio 1/MainEj1.java

MainEj1.java

```java
class Autor {
    String nombre;
    String nacionalidad;

    Autor(String nombre, String nacionalidad) {
        this.nombre = nombre;
        this.nacionalidad = nacionalidad;
    }
}
```

Figura 34. *Clase Autor (MainEj1.java)*

MainEj1.java

```java
class Libro {
    String titulo;
    int anio;
    Autor autor;

    Libro(String titulo, int anio, Autor autor) {
        this.titulo = titulo;
        this.anio = anio;
        this.autor = autor;
    }

    void mostrarInformacion() {
        System.out.println("Titulo: " + titulo);
        System.out.println("Año: " + anio);
        System.out.println("Autor: " + autor.nombre);
        System.out.println();
    }
}
```

Figura 35. *Clase Libro (MainEj1.java)*

MainEj1.java

```java
public class MainEj1 {
    public static void main(String[] args) {
        Autor autor1 = new Autor("Gabriel Garcia Marquez", "Colombiana");
        Autor autor2 = new Autor("Miguel de Cervantes", "Española");

        Libro libro1 = new Libro("Cien años de soledad", 1967, autor1);
        Libro libro2 = new Libro("Don Quijote", 1605, autor2);

        libro1.mostrarInformacion();
        libro2.mostrarInformacion();
    }
}
```

Figura 36. *Clase MainEj1 (MainEj1.java)*

Output - Ejercicios_APE_4 (run)

```
run:
Titulo: Cien años de soledad
Año: 1967
Autor: Gabriel Garcia Marquez

Titulo: Don Quijote
Año: 1605
Autor: Miguel de Cervantes
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 37. *Salida en consola, Grupo 3, ejercicio 1*

Libro guarda un objeto Autor completo y no solo el nombre en texto. En el main primero se crean los autores y luego se pasan al constructor de cada libro. Dentro de mostrarInformacion() se llega al nombre con autor.nombre.

Ejercicio 2: Rectángulo (firmas de métodos)

Enunciado: Crear la clase Rectangulo (base, altura) con los métodos calcularArea(), calcularPerimetro(), esCuadrado(), obtenerDescripcion() y mostrar(), usando el tipo de retorno correcto en cada firma.

Archivo: Grupo 3/Ejercicio 2/MainEj2.java

MainEj2.java

```java
class Rectangulo {
    double base;
    double altura;

    Rectangulo(double base, double altura) {
        this.base = base;
        this.altura = altura;
    }

    double calcularArea() {
        return base * altura;
    }

    double calcularPerimetro() {
        return 2 * (base + altura);
    }

    boolean esCuadrado() {
        return base == altura;
    }

    String obtenerDescripcion() {
        return "Rectangulo de " + base + " x " + altura;
    }

    void mostrar() {
        System.out.println(obtenerDescripcion());
        System.out.println("Area: " + calcularArea());
        System.out.println("Perimetro: " + calcularPerimetro());
        System.out.println("Es cuadrado: " + esCuadrado());
    }
}
```

Figura 38. *Clase Rectangulo (MainEj2.java)*

MainEj2.java

```java
public class MainEj2 {
    public static void main(String[] args) {
        Rectangulo r1 = new Rectangulo(4.0, 6.0);
        Rectangulo r2 = new Rectangulo(5.0, 5.0);

        r1.mostrar();
        System.out.println();
        r2.mostrar();
    }
}
```

Figura 39. *Clase MainEj2 (MainEj2.java)*

Output - Ejercicios_APE_4 (run)

```
run:
Rectangulo de 4.0 x 6.0
Area: 24.0
Perimetro: 20.0
Es cuadrado: false

Rectangulo de 5.0 x 5.0
Area: 25.0
Perimetro: 20.0
Es cuadrado: true
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 40. *Salida en consola, Grupo 3, ejercicio 2*

Área y perímetro devuelven double, esCuadrado() devuelve boolean, obtenerDescripcion() devuelve String y mostrar() es void porque solo imprime. Con 4 x 6 sale false y con 5 x 5 sale true.

Ejercicio 3: Producto y Venta (objeto como parámetro)

Enunciado: Crear la clase Producto (nombre, precio, stock) con reducirStock(int cantidad), que solo actúa si hay stock suficiente (si no, imprime "Stock insuficiente."), y mostrarProducto(). Crear la clase Venta (cantidad) con aplicar(Producto producto), que reduzca el stock y muestre el total a pagar.

Archivo: Grupo 3/Ejercicio 3/MainEj3.java

MainEj3.java

```java
class Producto {
    String nombre;
    double precio;
    int stock;

    Producto(String nombre, double precio, int stock) {
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    void reducirStock(int cantidad) {
        if (cantidad > 0 && stock >= cantidad) {
            stock = stock - cantidad;
        } else {
            System.out.println("Stock insuficiente.");
        }
    }

    void mostrarProducto() {
        System.out.println("Producto: " + nombre);
        System.out.println("Precio: $" + precio);
        System.out.println("Stock: " + stock);
    }
}
```

Figura 41. *Clase Producto (MainEj3.java)*

MainEj3.java

```java
class Venta {
    int cantidad;

    Venta(int cantidad) {
        this.cantidad = cantidad;
    }

    void aplicar(Producto producto) {
        if (cantidad > 0 && producto.stock >= cantidad) {
            producto.reducirStock(cantidad);
            double total = producto.precio * cantidad;
            System.out.println("Total a pagar: $" + total);
            producto.mostrarProducto();
        } else {
            System.out.println("Stock insuficiente.");
        }
    }
}
```

Figura 42. *Clase Venta (MainEj3.java)*

MainEj3.java

```java
public class MainEj3 {
    public static void main(String[] args) {
        Producto p1 = new Producto("Teclado", 15.0, 10);

        Venta v1 = new Venta(3);
        v1.aplicar(p1);

        System.out.println();

        Venta v2 = new Venta(20);
        v2.aplicar(p1);
    }
}
```

Figura 43. *Clase MainEj3 (MainEj3.java)*

```
Output - Ejercicios_APE_4 (run)
run:
Total a pagar: $45.0
Producto: Teclado
Precio: $15.0
Stock: 7

Stock insuficiente.
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 44. *Salida en consola, Grupo 3, ejercicio 3*

aplicar() recibe el producto como parámetro y le baja el stock. La primera venta de 3 teclados da $45.0 y el stock queda en 7. La segunda pide 20, así que no se hace y sale Stock insuficiente.

Ejercicio 4: Transferencias (objeto como parámetro)
Enunciado: Usar la clase Cuenta y agregar el método transferir(Cuenta destino, double monto), que retire el monto de la cuenta actual y lo deposite en la cuenta destino solo si hay fondos. En main, crear 2 cuentas, hacer una transferencia válida y otra que falle.
Archivo: Grupo 3/Ejercicio 4/MainEj4.java

MainEj4.java

```java
class Cuenta {
    String titular;
    double saldo;

    Cuenta(String titular, double saldo) {
        this.titular = titular;
        this.saldo = saldo;
    }

    void transferir(Cuenta destino, double monto) {
        if (monto > 0 && saldo >= monto) {
            saldo = saldo - monto;
            destino.saldo = destino.saldo + monto;

            System.out.println("Transferencia exitosa.");
        } else {
            System.out.println("Transferencia fallida: fondos insuficientes o monto invalido.");
        }
    }

    void mostrar() {
        System.out.println("Titular: " + titular);
        System.out.println("Saldo: $" + saldo);
    }
}
```

Figura 45. *Clase Cuenta (MainEj4.java)*

MainEj4.java

```java
public class MainEj4 {
    public static void main(String[] args) {
        Cuenta cuenta1 = new Cuenta("Carlos", 100.0);
        Cuenta cuenta2 = new Cuenta("Pedro", 50.0);

        cuenta1.transferir(cuenta2, 30.0);

        System.out.println();

        cuenta1.transferir(cuenta2, 200.0);

        System.out.println();
        cuenta1.mostrar();
        cuenta2.mostrar();
    }
}
```

Figura 46. *Clase MainEj4 (MainEj4.java)*

```
Output - Ejercicios_APE_4 (run)
run:
Transferencia exitosa.

Transferencia fallida: fondos insuficientes o monto invalido.

Titular: Carlos
Saldo: $70.0
Titular: Pedro
Saldo: $80.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 47. *Salida en consola, Grupo 3, ejercicio 4*

transferir() recibe la cuenta destino como objeto y solo mueve el dinero si el monto es positivo y alcanza el saldo. Carlos transfiere $30 a Pedro sin problema, pero la de $200 falla. Al final Carlos queda con $70 y Pedro con $80.

Ejercicio 5: Biblioteca (colaboración entre clases)

Enunciado: Crear tres clases que colaboren entre sí: Usuario, Libro y Prestamo. Prestamo tiene atributos de tipo Usuario y Libro, y su método registrar() intenta prestar el libro y muestra un mensaje según el resultado. En main, hacer que dos usuarios intenten pedir el mismo libro.

Archivo: Grupo 3/Ejercicio 5/MainEj5.java

MainEj5.java

```java
class Usuario {
    String nombre;
    String cedula;

    Usuario(String nombre, String cedula) {
        this.nombre = nombre;
        this.cedula = cedula;
    }
}
```

Figura 48. *Clase Usuario (MainEj5.java)*

MainEj5.java

```java
class Libro {
    String titulo;
    boolean disponible;

    Libro(String titulo) {
        this.titulo = titulo;
        this.disponible = true;
    }

    boolean prestar() {
        if (disponible) {
            disponible = false;
            return true;
        }

        return false;
    }

    void devolver() {
        disponible = true;
        System.out.println("Libro devuelto correctamente.");
    }
}
```

Figura 49. *Clase Libro (MainEj5.java)*

MainEj5.java

```java
class Prestamo {
    Usuario usuario;
    Libro libro;

    Prestamo(Usuario usuario, Libro libro) {
        this.usuario = usuario;
        this.libro = libro;
    }

    void registrar() {
        if (libro.prestar()) {
            System.out.println("Prestamo registrado correctamente.");
            System.out.println("Usuario: " + usuario.nombre);
            System.out.println("Libro: " + libro.titulo);
        } else {
            System.out.println("El libro no esta disponible para " + usuario.nombre);
        }

        System.out.println();
    }
}
```

Figura 50. *Clase Prestamo (MainEj5.java)*

MainEj5.java

```java
public class MainEj5 {
    public static void main(String[] args) {
        Usuario usuario1 = new Usuario("Carlos", "1801234567");
        Usuario usuario2 = new Usuario("Maria", "1807654321");

        Libro libro1 = new Libro("Java para principiantes");

        Prestamo prestamo1 = new Prestamo(usuario1, libro1);
        Prestamo prestamo2 = new Prestamo(usuario2, libro1);

        prestamo1.registrar();
        prestamo2.registrar();
    }
}
```

Figura 51. *Clase MainEj5 (MainEj5.java)*

Output - Ejercicios_APE_4 (run)

```
run:
Prestamo registrado correctamente.
Usuario: Carlos
Libro: Java para principiantes

El libro no esta disponible para Maria
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figura 52. *Salida en consola, Grupo 3, ejercicio 5*

Los dos préstamos usan el mismo objeto libro1. Cuando Carlos lo pide, prestar() cambia disponible a false, por eso cuando María lo intenta sale que el libro no está disponible.

2.7 Resultados obtenidos

Los 18 programas compilaron y corrieron sin errores. En la Tabla 2 se resume lo que se probó en cada ejercicio y lo que salió en consola; las capturas están en la sección anterior y en los anexos.

Tabla 2. *Resumen de pruebas realizadas*

Ejercicio	Datos de prueba	Resultado obtenido
G1-1 Notas del curso	5 estudiantes con 3 notas cada uno	Valeria 8.83 Aprobado; total de aprobados: 3
G1-2 Inventario	Venta de 3 esferos y 3 borradores	Borrador y Carpeta con Reponer; total $54.25
G1-3 Veterinaria	Mascota m4 = m1 y m4.edad + 1	m1.edad pasa de 4 a 5
G1-4 Cuenta de ahorro	Saldos 200 y 350, depósitos de 50	6 meses y 3 meses
G1-5 Biblioteca	Camila pide 2 libros, Diego 1	Queda 1 libro disponible

G2-1 Tipos de retorno	Base 4.5, altura 3.0	13.5
G2-2 Bienvenida	Antony, 19 años	Hola Antony, tienes 19 años...
G2-3 Parámetros	multiplicar(4.0, 5.5)	22.0
G2-4 Paso por valor	numero = 10	Se imprime 10
G2-5 Promedio	Notas 8, 9 y 10	9.0
G2-6 Validar edad	Edad 19	Edad válida: true
G2-7 Clase Producto	Stock 10, precio 25.5	255.0
G2-8 Instanciación	Objeto p con los mismos datos	Total del inventario: 255.0
G3-1 Libro y Autor	Cien años de soledad y Don Quijote	Título, año y autor de cada libro
G3-2 Rectángulo	4.0 x 6.0 y 5.0 x 5.0	Áreas 24.0 y 25.0; false y true
G3-3 Producto y Venta	Teclado $15, stock 10; ventas de 3 y 20	Total $45.0, stock 7; luego Stock insuficiente.
G3-4 Transferencias	Carlos $100, Pedro $50; 30 y 200	Primera exitosa, segunda falla; $70 y $80
G3-5 Préstamos	Carlos y María piden el mismo libro	Solo Carlos lo obtiene

2.8 Habilidades blandas empleadas en la práctica

☐Liderazgo
☒Trabajo en equipo
☒Comunicación asertiva
☐La empatía
☒Pensamiento crítico
☐Flexibilidad
☐La resolución de conflictos
☐Adaptabilidad
☒Responsabilidad

2.9 Conclusiones

- Tener un objeto como atributo, como el Autor dentro de Libro o el Usuario y el Libro dentro de Prestamo, evita repetir datos. El autor se crea una vez y lo puede usar cualquier libro.
- Escoger bien el tipo de retorno hace que el método sirva para más cosas. Como calcularArea() devuelve el valor en lugar de imprimirlo, en el main pudimos sumar las áreas de los dos rectángulos sin tocar la clase.
- Cuando un objeto se pasa como parámetro, el método trabaja sobre ese mismo objeto y no sobre una copia. Por eso Venta pudo bajar el stock del producto y transferir() pudo cambiar el saldo de la otra cuenta.
- No es lo mismo pasar un int que pasar un objeto. En el ejercicio de paso por valor la variable del main no cambió, pero en la veterinaria m1 sí cambió porque m4 era la misma mascota.
- Poner las validaciones dentro de los métodos (stock, fondos, disponibilidad, edad) evitó errores como saldos negativos, y el main quedó más corto.

2.10 Recomendaciones

- Antes de programar, anotar qué clases hay y qué atributos lleva cada una. En el ejercicio 5 esto nos ayudó a no mezclar lo que le toca a Libro con lo que le toca a Prestamo.
- Probar también el caso que tiene que fallar (stock insuficiente, transferencia sin fondos, libro ya prestado) y no solo el caso que funciona.

- Si dos ejercicios tienen clases con el mismo nombre, probarlos por separado o ponerlos en paquetes distintos.
- Al subir archivos a GitHub, revisar que el nombre coincida con la clase pública; Calculadora (1).java no compila hasta renombrarlo.
- Hacer git pull antes de subir cambios al repositorio, así no hay conflictos entre los integrantes.

2.11 Referencias bibliográficas

[1] M. I. Barquilla Galeano, Java programación: teoría y ejemplos, 1.ª ed. Madrid, España: RA-MA Editorial, 2025. [En línea]. Disponible en: https://elibro.net/es/lc/uta/titulos/285282

[2] F. Blasco, Programación orientada a objetos en Java, 1.ª ed. Bogotá, Colombia: Ediciones de la U, 2019. [En línea]. Disponible en: https://elibro.net/es/lc/uta/titulos/127125

[3] M. D. Contreras Castro, Lógica de programación a partir del pensamiento lógico con software DFD, PSeInt y LPP, 1.ª ed. Valencia, España: Ediciones USTA, 2023. [En línea]. Disponible en: https://elibro.net/es/lc/uta/titulos/288308

[4] J. M. Dueñas Meza, J. J. Valdés Benjumea y M. C. Rodríguez Niño, Didáctica para el aprendizaje de algoritmos, 1.ª ed. Bogotá, Colombia: Sello Editorial UNAD, 2020. [En línea]. Disponible en: https://elibro.net/es/lc/uta/titulos/288503

2.12 Anexos

Repositorio de GitHub:

https://github.com/brishy769-coder/SW-AyLP-APE-04-Clases-Objetos-y-Metodos

En el repositorio están los 18 ejercicios en las carpetas Grupo 1, Grupo 2 y Grupo 3, con el código de cada uno, las capturas de ejecución y las infografías. A continuación se incluyen esas capturas, tomadas por cada integrante al probar su código, y las infografías de cada grupo.

Anexo 1. Ejecución del Grupo 1, ejercicio 1 (captura del repositorio)

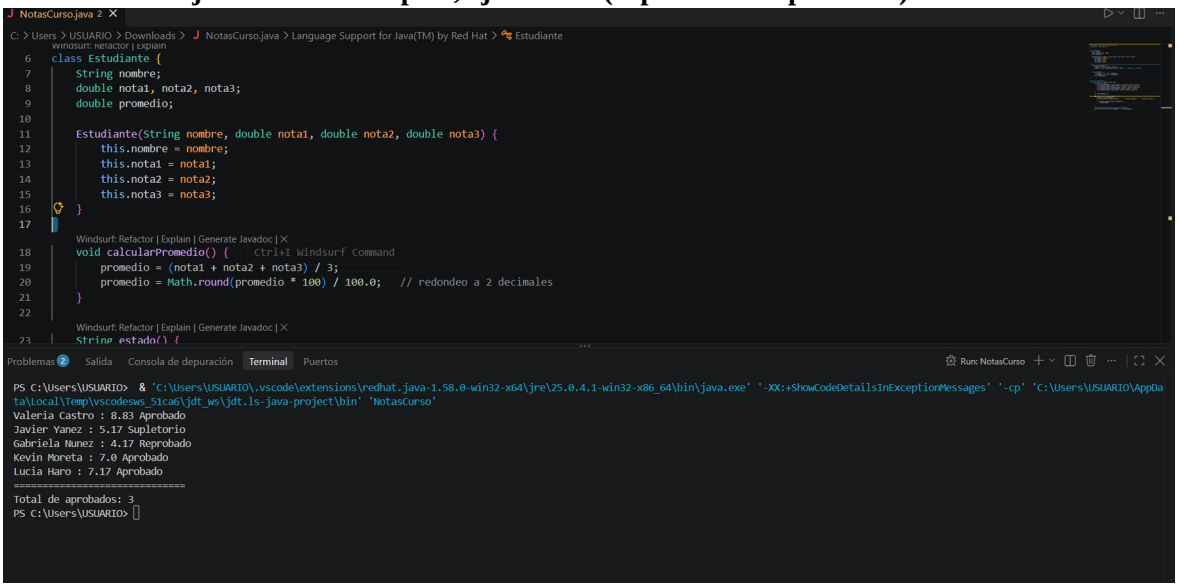


Figura 53. *Ejecución del Grupo 1, ejercicio 1 (captura del repositorio)*

Anexo 2. Ejecución del Grupo 1, ejercicio 2 (captura del repositorio)

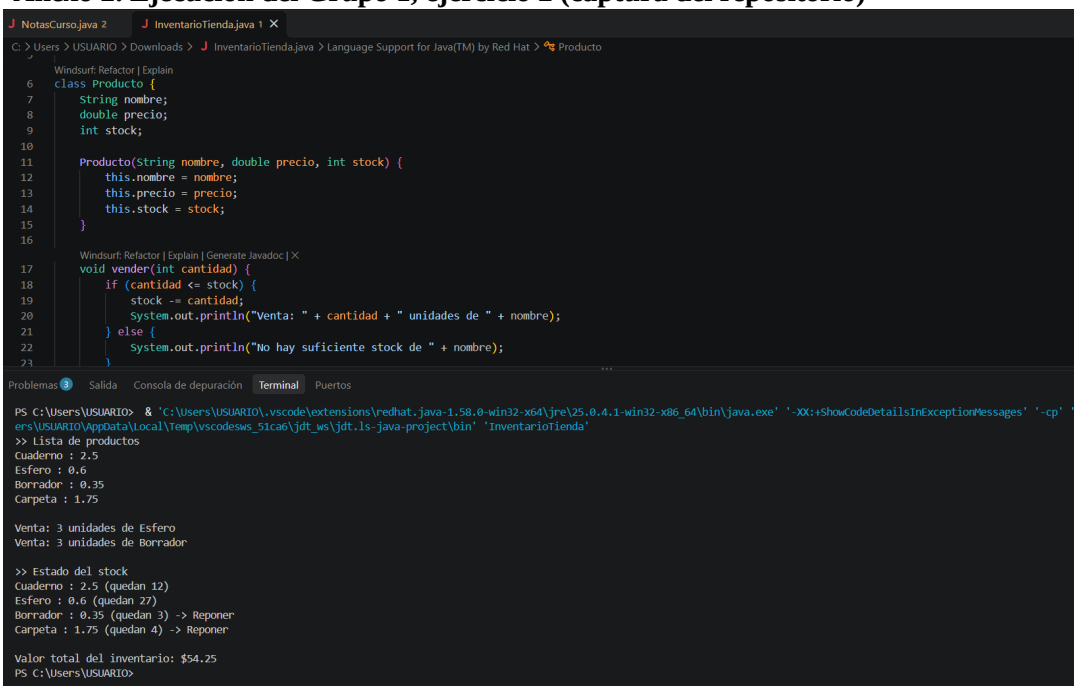


Figura 54. *Ejecución del Grupo 1, ejercicio 2 (captura del repositorio)*

Anexo 3. Ejecución del Grupo 1, ejercicio 3 (captura del repositorio)

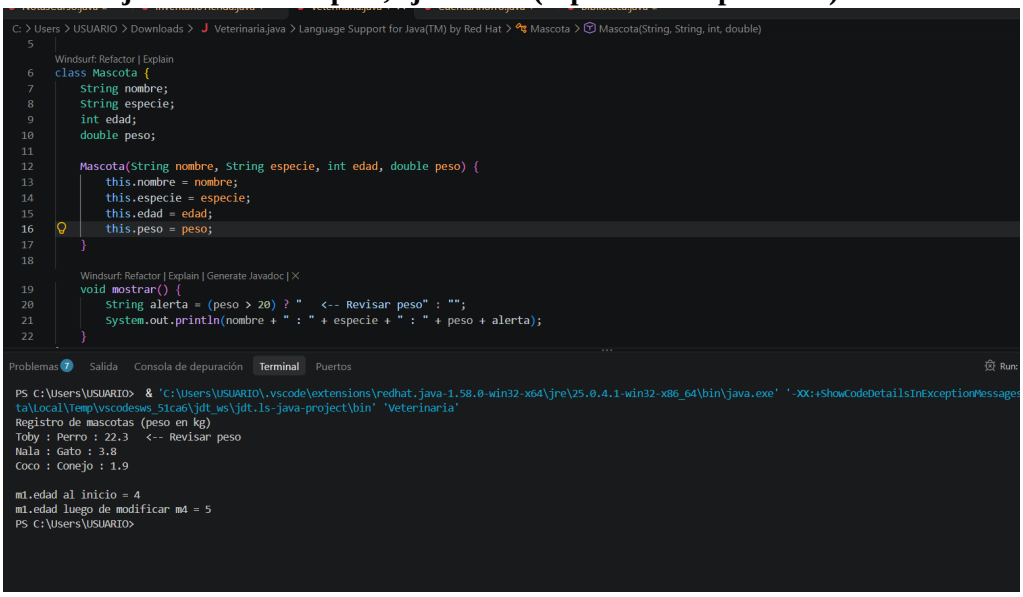


Figura 55. *Ejecución del Grupo 1, ejercicio 3 (captura del repositorio)*

Anexo 4. Ejecución del Grupo 1, ejercicio 4 (captura del repositorio)

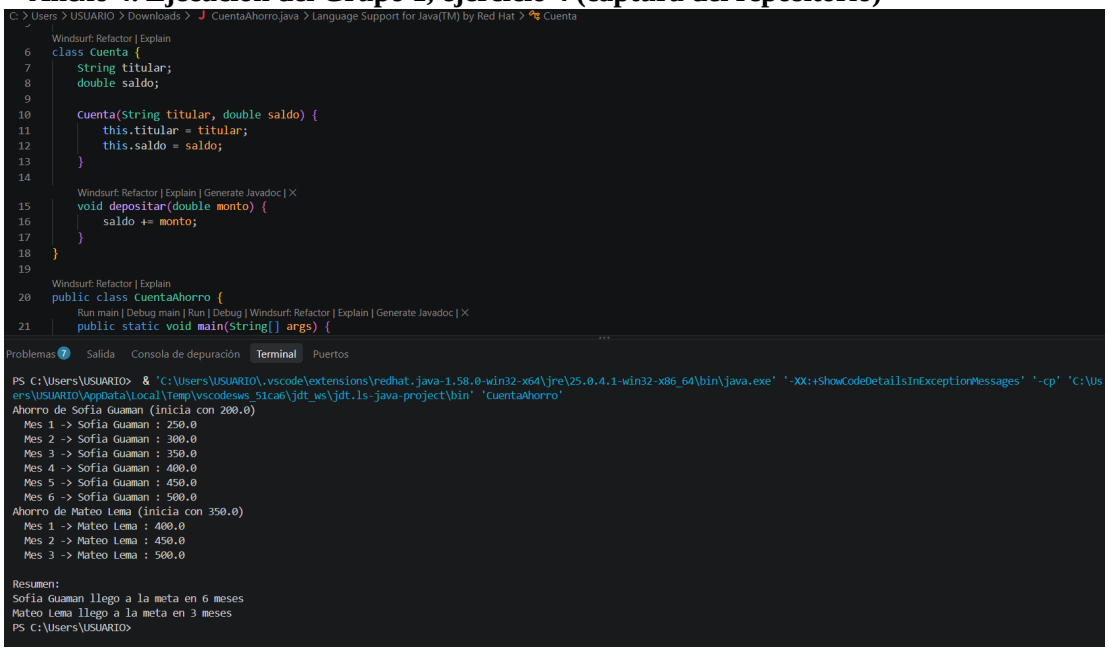


Figura 56. *Ejecución del Grupo 1, ejercicio 4 (captura del repositorio)*

Anexo 5. Ejecución del Grupo 1, ejercicio 5 (captura del repositorio)

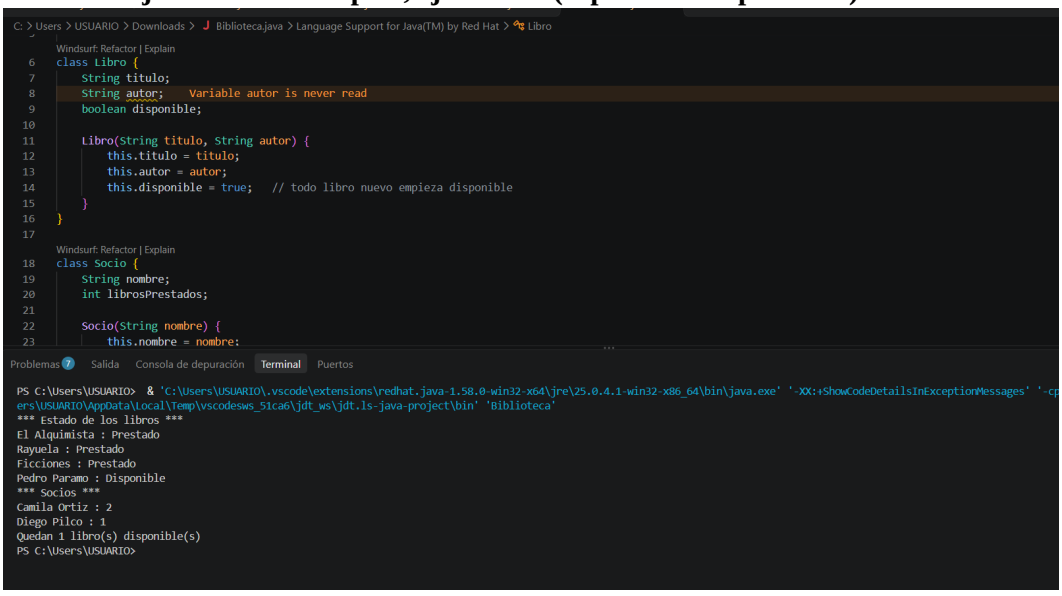


Figura 57. *Ejecución del Grupo 1, ejercicio 5 (captura del repositorio)*

Anexo 6. Ejecución del Grupo 2, ejercicio 1 (captura del repositorio)

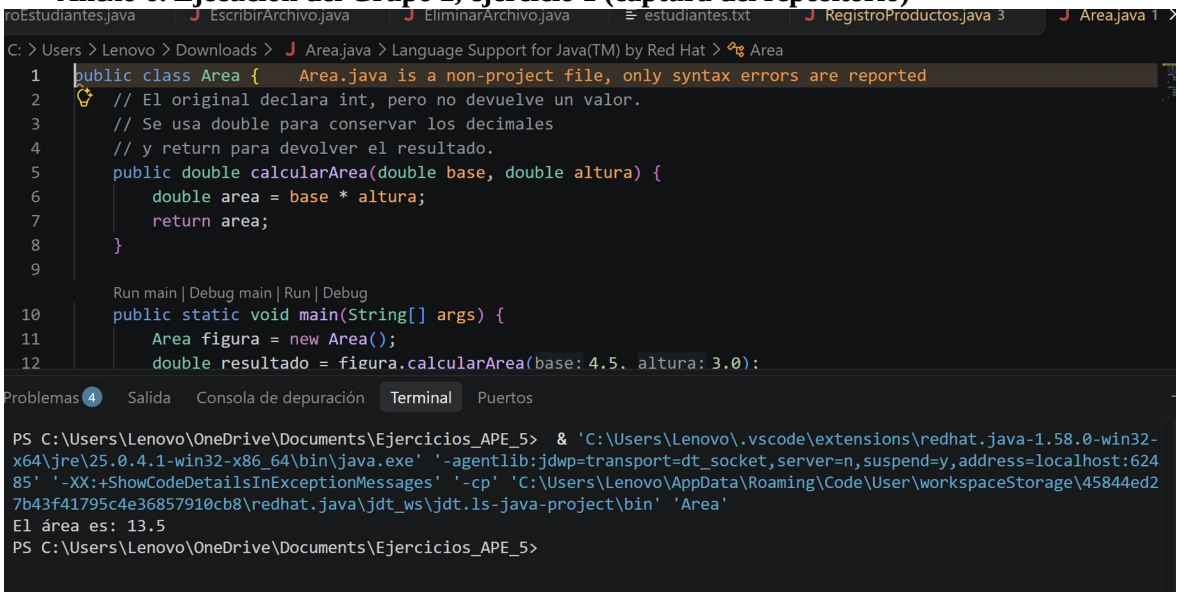


Figura 58. *Ejecución del Grupo 2, ejercicio 1 (captura del repositorio)*

Anexo 7. Ejecución del Grupo 2, ejercicio 2 (captura del repositorio)

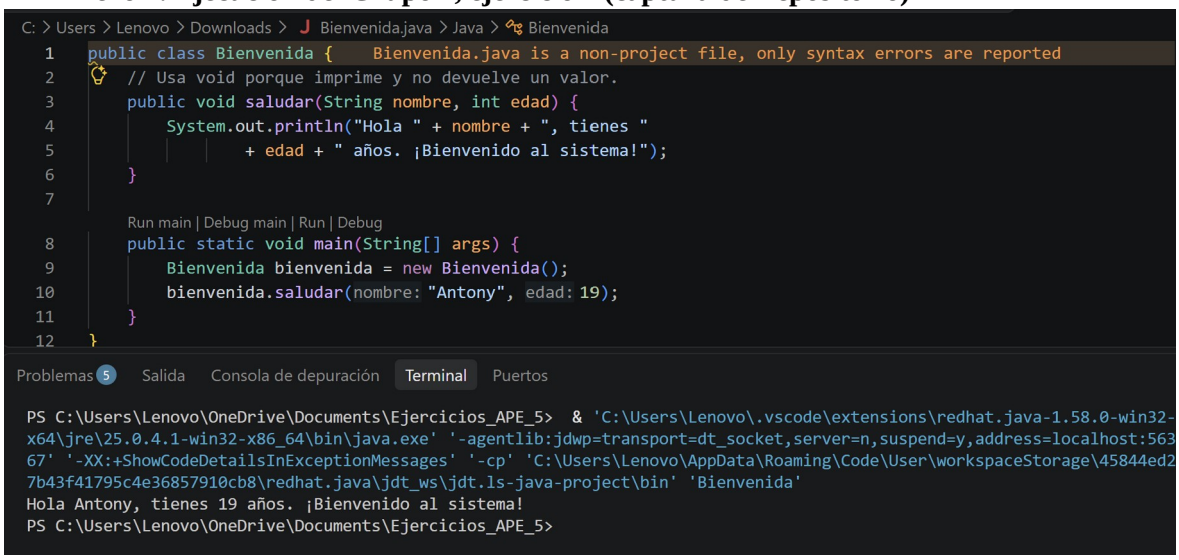


Figura 59. *Ejecución del Grupo 2, ejercicio 2 (captura del repositorio)*

Anexo 8. Ejecución del Grupo 2, ejercicio 8 (captura del repositorio)

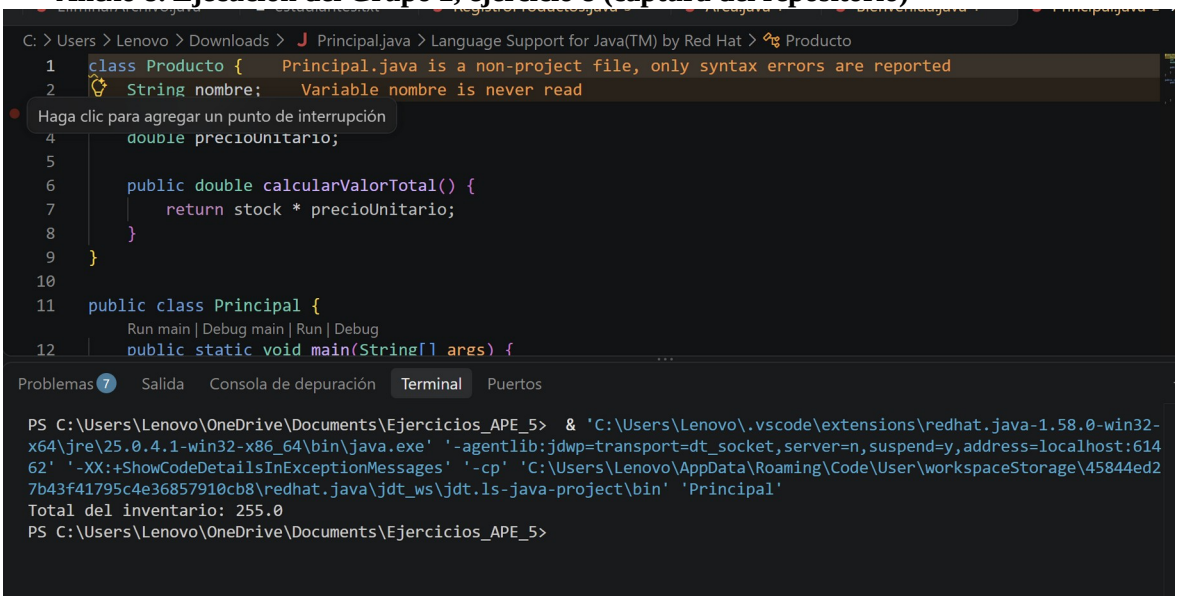


Figura 60. *Ejecución del Grupo 2, ejercicio 8 (captura del repositorio)*

Anexo 9. Ejecución del Grupo 3, ejercicio 1 (captura del repositorio)

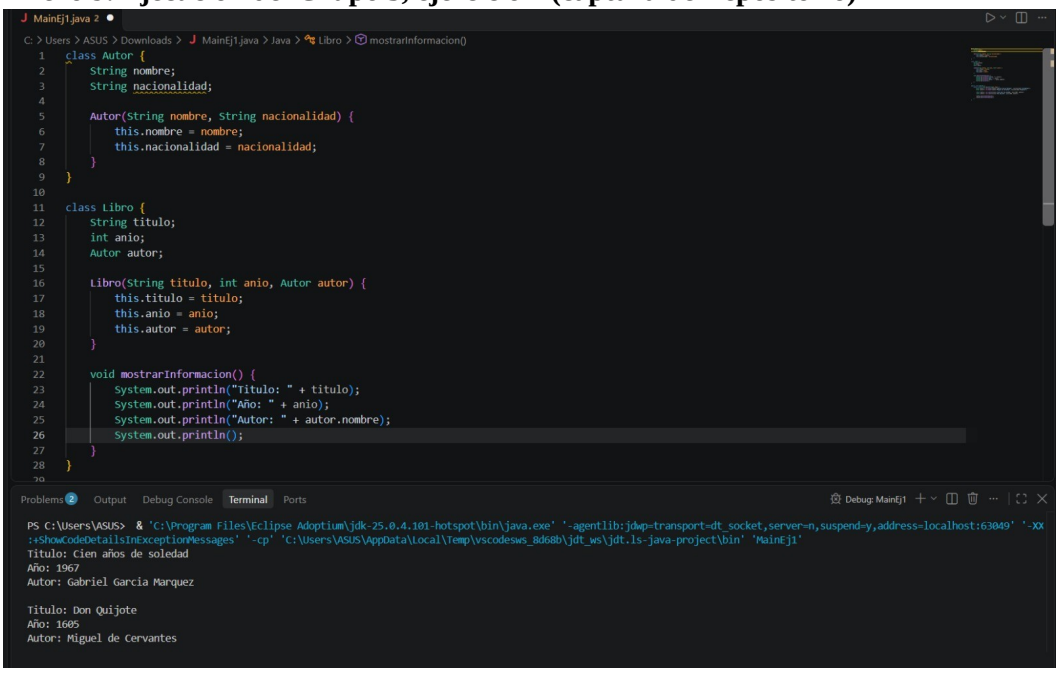


Figura 61. *Ejecución del Grupo 3, ejercicio 1 (captura del repositorio)*

Anexo 10. Ejecución del Grupo 3, ejercicio 2 (captura del repositorio)

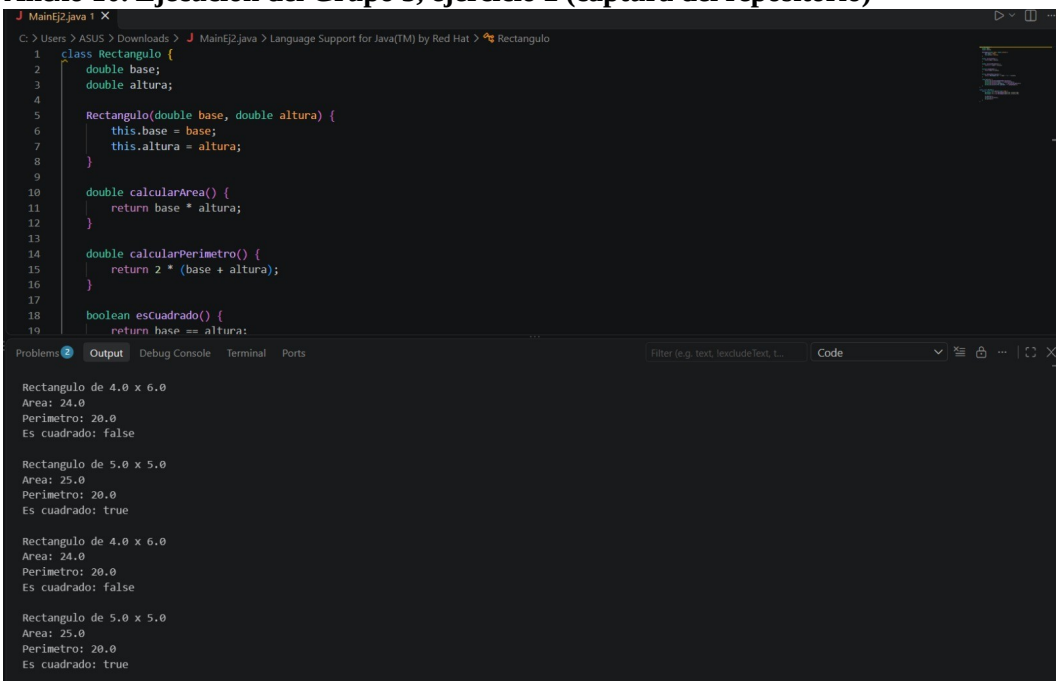


Figura 62. *Ejecución del Grupo 3, ejercicio 2 (captura del repositorio)*

Anexo 11. Ejecución del Grupo 3, ejercicio 3 (captura del repositorio)

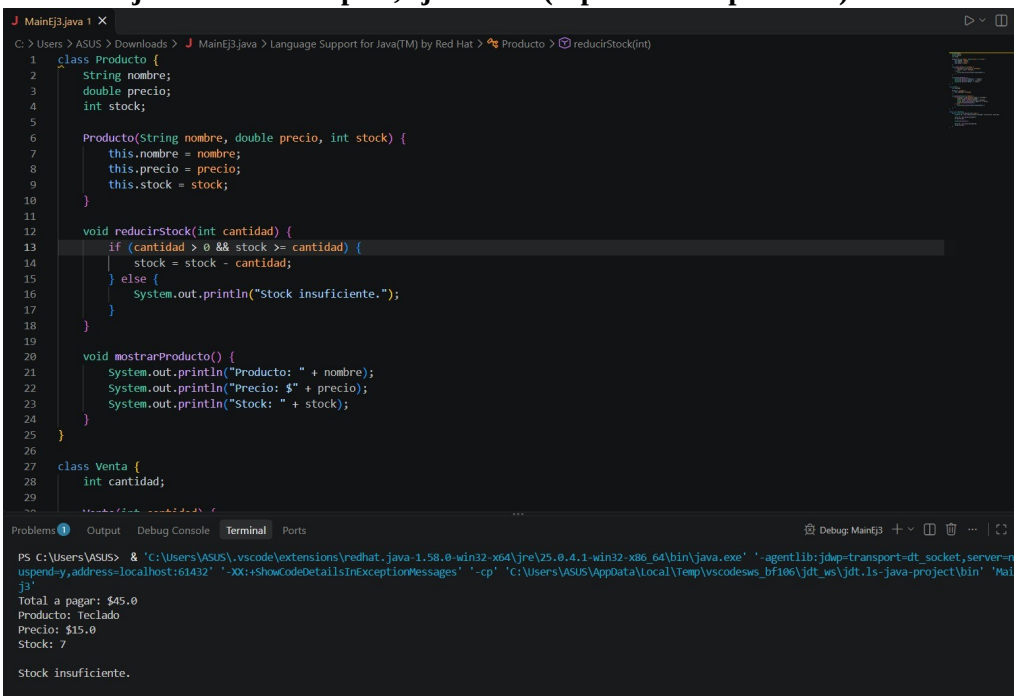


Figura 63. *Ejecución del Grupo 3, ejercicio 3 (captura del repositorio)*

Anexo 12. Ejecución del Grupo 3, ejercicio 4 (captura del repositorio)

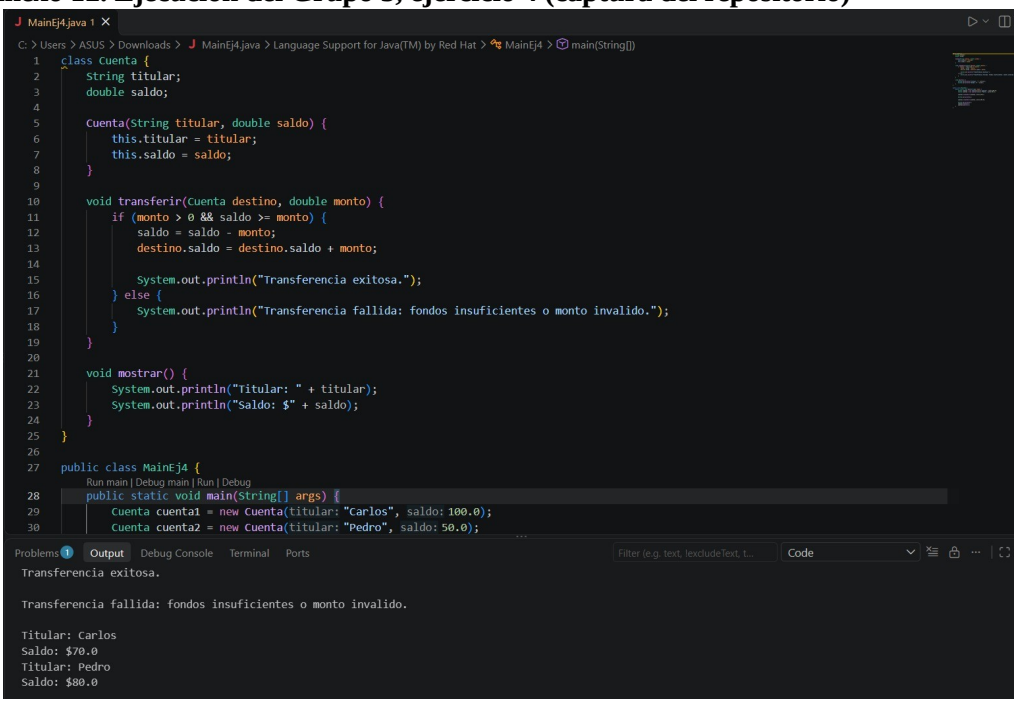


Figura 64. *Ejecución del Grupo 3, ejercicio 4 (captura del repositorio)*

Anexo 13. Ejecución del Grupo 3, ejercicio 5 (captura del repositorio)

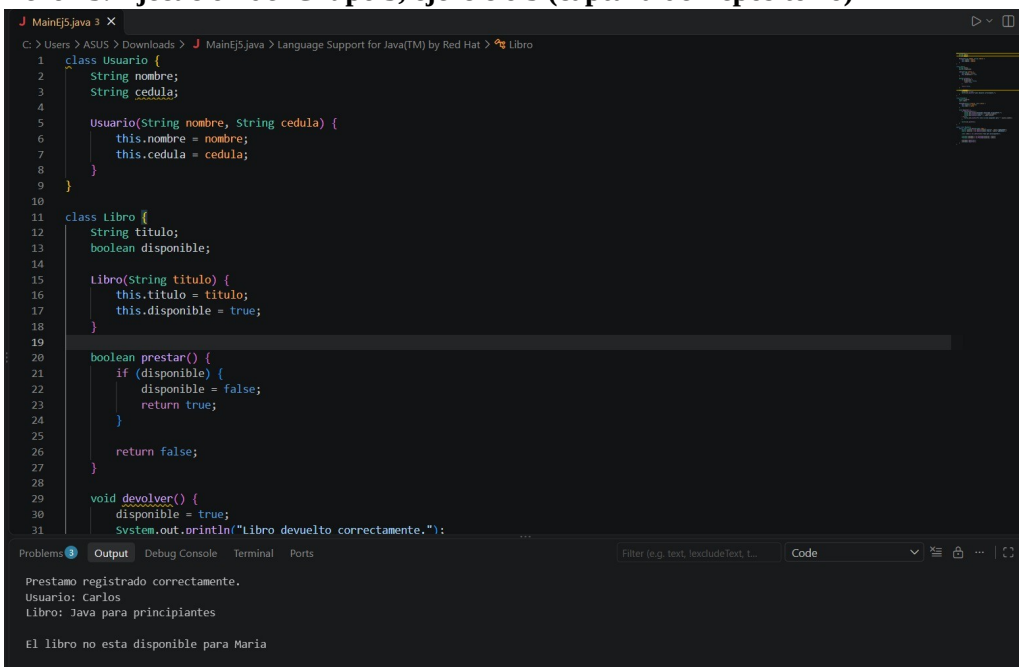


Figura 65. *Ejecución del Grupo 3, ejercicio 5 (captura del repositorio)*

Anexo 14. Infografía del Grupo 1



Figura 66. *Infografía del Grupo 1*

Anexo 15. Infografía del Grupo 2



Figura 67. *Infografía del Grupo 2*

Anexo 16. Infografía del Grupo 3

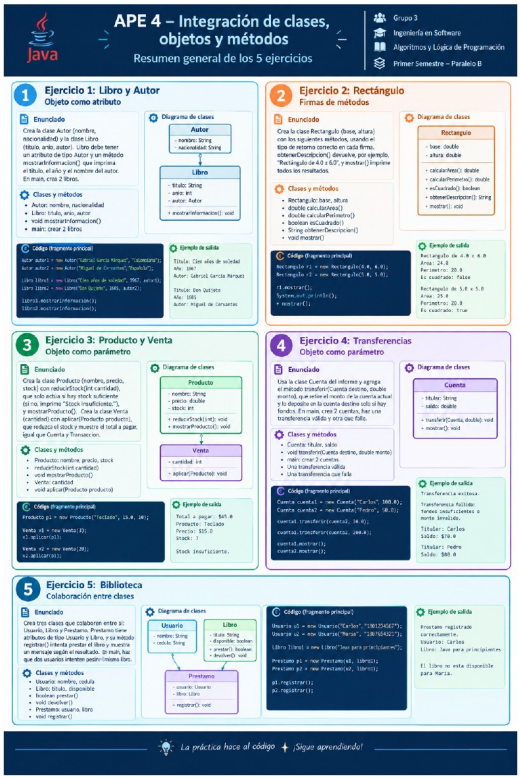


Figura 68. *Infografía del Grupo 3*